

API REST Supermercado - IaC EKS

Marlon Ernesto Figueroa Fuentes

Índice

API REST Supermercado — Documentación técnica	2
1. Control del documento	2
2. Resumen ejecutivo	2
3. Introducción	3
4. Objetivos y alcance	4
5. Requisitos	6
6. Decisiones de arquitectura	7
7. Arquitectura lógica	8
8. Arquitectura de red	9
9. Recorrido de una petición	12
10. Parámetros globales (<code>vars.yaml</code>)	13
11. Stacks CloudFormation	14
12. Amazon VPC	16
13. AWS KMS	16
14. Amazon DynamoDB	17
15. AWS IAM	18
16. Amazon EKS	19
17. Stack auxiliar EC2 y SSM	22
18. Amazon API Gateway	22
19. Plataforma Kubernetes	23
20. Karpenter	24
21. Istio	26
22. Microservicio FastAPI	28
23. Contrato de la API	30
24. Seguridad e identidad	32
25. Pipeline de despliegue	33
26. Operación diaria	34
27. Pruebas	35
28. Costos en el laboratorio	36
29. Incidentes frecuentes	37
30. Riesgos y trabajo futuro	38
31. Estructura del repositorio	39
32. Generación de este PDF	40
33. Glosario	40
34. Anexo — lectura de un despliegue correcto	41
35. Referencias	42

API REST Supermercado — Documentación técnica

Autor: Marlon Ernesto Figueroa Fuentes **Versión del documento:** 2.0 **Fecha:** 27 de septiembre de 2026 **Entorno descrito:** laboratorio `dev` (región configurable, por defecto `us-east-1`)

Infraestructura como código (IaC) para una API REST de supermercado en AWS. El tráfico entra por **Amazon API Gateway**, atraviesa un **VPC Link** hacia un **Network Load Balancer interno**, pasa por **Istio** y llega a un microservicio **FastAPI** en **Amazon EKS**. Los datos persisten en **Amazon DynamoDB**. El cómputo elástico de cargas de trabajo usa **Karpenter** con instancias **Spot**. La red, la identidad, el cifrado y el clúster se declaran en plantillas **CloudFormation** modulares.

Este informe describe el diseño, el recorrido de una petición, cada stack, la plataforma Kubernetes, el contrato de la API, la operación y la forma de regenerar el PDF. Los identificadores de cuenta, ARN de balanceadores y nombres DNS reales no se copian aquí: viven en `vars.yaml` después del despliegue y no deben publicarse en documentación compartida.

1. Control del documento

Campo	Valor
Título	API REST Supermercado — Infraestructura en AWS
Autor	Marlon Ernesto Figueroa Fuentes
Repositorio	<code>iac_api_mercado</code>
Fuente editable	<code>docs/PROYECTO.md</code>
Salida	<code>docs/PROYECTO.pdf</code> mediante <code>make docs-pdf</code>
Diagramas	<code>docs/diagramas/*.mmd</code> renderizados a <code>docs/media/*.png</code>
Audiencia	Quien despliega, opera o evalúa la solución

El PDF se genera con Pandoc y un motor LaTeX (Tectonic, XeLaTeX o pdfLaTeX). Las figuras se rasterizan con Mermaid CLI sobre fondo blanco, con el lado mayor acotado, y se les asigna una densidad para que el tamaño impreso no pase de unos 14,5 cm de ancho ni de unos 11 cm de alto. Pandoc solo reduce una figura si aun así no cabe; no la estira para llenar la página.

2. Resumen ejecutivo

La solución expone dos recursos de negocio, **productos** y **clientes**, mediante una API HTTP regional. El cliente público no habla con los pods: habla con API Gateway. El balanceador que recibe el tráfico desde la VPC es interno, así que los nodos y el proceso de la aplicación permanecen en subnets privadas.

Aspecto	Descripción
Dominio	Catálogo de productos y registro de clientes
Exposición	API Gateway REST, stage configurable (<code>dev</code> por defecto)
Runtime	Python FastAPI 0.115 y boto3 sobre EKS
Persistencia	Dos tablas DynamoDB en modo bajo demanda
IaC	Plantillas CloudFormation por capa y manifiestos Kubernetes
Cómputo	Un node group bootstrap más nodos Spot de Karpenter
Identidad de la app	IRSA: el pod asume un rol IAM sin llaves estáticas
Entorno objetivo	Desarrollo y laboratorio

Rutas públicas, ya con el nombre del stage delante:

- `/{{stage}}/productos` — alta, consulta, actualización y baja de productos
- `/{{stage}}/clientes` — alta, consulta, actualización y baja de clientes

El stage no se reenvía al balanceador. API Gateway traduce `/{{stage}}/productos` a `http://{{dns-del-nlb}}/productos`. Dentro del clúster, un VirtualService de Istio entrega ese prefijo al Service `fastapi` del namespace `api-mercado`, puerto 8000.

El diseño privilegia costo de laboratorio frente a alta disponibilidad completa: un solo NAT Gateway, DynamoDB on-demand, capacidad Spot, un nodo bootstrap pequeño y una réplica de la API cuando se despliega desde ConfigMap. Esas decisiones están declaradas en `vars.yaml` y se explican en las secciones de arquitectura y de costos.

3. Introducción

3.1 Contexto

Un supermercado necesita consultar y mantener un catálogo de productos (nombre, precio y existencias) y un registro sencillo de clientes (nombre y fecha de nacimiento). El volumen de un laboratorio no justifica un servidor de bases de datos siempre encendido ni una flota fija de instancias. Sí necesita una frontera HTTP estable, una red privada para el cómputo y una forma repetible de crear el entorno desde cero.

El repositorio concentra esa repetición en plantillas y scripts. Quien clona el proyecto edita `vars.yaml`, ejecuta `make all` y obtiene la misma topología: VPC, clave KMS, tablas, roles, clúster EKS, reglas auxiliares, plataforma (Karpenter e Istio) y la API.

3.2 Problema que resuelve

Sin una frontera y sin identidad de workload, lo habitual en un prototipo es publicar un balanceador a Internet y guardar llaves de acceso en variables del contenedor. Ese atajo mezcla tres riesgos: la superficie de ataque crece, las credenciales se filtran con la imagen o con el manifiesto, y el entorno no se puede reconstruir igual en otra cuenta.

Este proyecto separa esas responsabilidades. La única URL pública de aplicación es la de API Gateway. El Network Load Balancer es interno. El pod obtiene credenciales temporales porque su ServiceAccount está anotado con un rol IAM federado al OIDC del clúster. La red, los roles y las tablas se crean por CloudFormation en un orden fijo, de modo que cada stack recibe los identificadores del anterior.

3.3 Qué contiene este informe

El informe sigue el orden en que se razona la solución y después el orden en que se despliega. Primero están el alcance, los requisitos y las decisiones. Luego la vista lógica, la vista de red y el recorrido de una petición, con los diagramas correspondientes. A continuación se detalla cada plantilla de CloudFormation, la plataforma Kubernetes y el código de FastAPI. El cierre cubre seguridad, el pipeline, las pruebas, la operación, los costos, los fallos habituales y la generación del propio PDF.

4. Objetivos y alcance

4.1 Objetivo general

Desplegar una API REST de supermercado sobre AWS, reproducible desde código, con el plano de datos en una VPC privada y con acceso a DynamoDB mediante roles de IAM asociados a cuentas de servicio de Kubernetes.

4.2 Objetivos específicos

- Declarar la red, el cifrado de secretos de EKS, las tablas, los roles y el clúster en plantillas CloudFormation independientes, encadenadas por salidas.
- Instalar Karpenter e Istio sobre un node group bootstrap mínimo y dejar que Karpenter agregue nodos Spot cuando haya pods en estado Pending.
- Publicar productos y clientes solo a través de API Gateway y un VPC Link hacia un NLB interno.
- Ejecutar FastAPI sin credenciales estáticas, montando el código desde un ConfigMap cuando no hay una imagen en ECR.
- Dejar un comando único (`make all`) y comandos parciales para repetir cada fase.
- Documentar la arquitectura de forma que el PDF se regenere a partir de `docs/PROYECTO.md` y de los diagramas Mermaid.

4.3 Alcance incluido

- VPC con dos subnets públicas, dos privadas, un Internet Gateway y un NAT Gateway.
- Clave KMS con rotación para el cifrado de secretos del clúster.
- Tablas DynamoDB `productos` y `clientes` en modo `PAY_PER_REQUEST`.
- Roles del plano de control, de los nodos bootstrap, de los nodos Karpenter, la policy del controlador Karpenter y la policy de datos de FastAPI.
- Clúster EKS con endpoint público y privado, grupo de nodos bootstrap Amazon Linux 2023, proveedor OIDC, roles IRSA y addons `vpc-cni`, `kube-proxy` y `CoreDNS`.
- Grupo de seguridad auxiliar para el camino NLB hacia Istio y parámetros en SSM.
- API Gateway REST regional, VPC Link, recursos `/productos` y `/clientes`, stage configurable.
- Manifiestos de Karpenter (`EC2NodeClass`, `NodePool`), IstioOperator reducido, Gateway, Service del NLB interno y la aplicación.
- Scripts de despliegue, descubrimiento del NLB, render de manifiestos y generación de este PDF.

4.4 Fuera de alcance en esta versión

- Canal de integración y despliegue continuo.
- Registro ECR obligatorio y promoción de imágenes entre entornos.
- Multi-región, cuentas múltiples y AWS Organizations.
- WAF, throttling fino, API keys, JWT o Amazon Cognito delante de la API.
- Host bastión, VPN de administración y malla de observabilidad (X-Ray, Managed Prometheus, tableros).
- Índices secundarios, paginación de `Scan` y modelo de pedidos o inventario avanzado.
- Alta disponibilidad del NAT (un NAT por zona) y nodos On-Demand de producción.

4.5 Supuestos y restricciones

- Existe una cuenta AWS con cuota para un clúster EKS, un NAT Gateway y un VPC Link.
- La CLI de AWS está autenticada. El perfil se indica en `AwsProfile` dentro de `vars.yaml`.
- La versión de Kubernetes objetivo es 1.33 o la que indique `KubernetesVersion`. En 1.33 los nodos deben usar Amazon Linux 2023; Amazon Linux 2 no ofrece AMI válida para esa versión en EKS.
- El laboratorio acepta interrupciones de Spot y una sola réplica de la API en el modo Config-Map.
- La API queda abierta (`AuthorizationType: NONE`). Eso es aceptable solo en un stage de laboratorio.

5. Requisitos

5.1 Requisitos funcionales

ID	Requisito	Dónde se cumple
RF-01	Crear un producto con nombre, precio y stock	POST <code>/productos</code>
RF-02	Listar productos	GET <code>/productos</code>
RF-03	Consultar un producto por identificador	GET <code>/productos/{id}</code>
RF-04	Actualizar un producto	PUT <code>/productos/{id}</code>
RF-05	Eliminar un producto	DELETE <code>/productos/{id}</code>
RF-06	Crear un cliente con nombre y fecha de nacimiento	POST <code>/clientes</code>
RF-07	Listar, consultar, actualizar y eliminar clientes	Rutas <code>/clientes</code> análogas
RF-08	Exponer un health check para sondas	GET <code>/health</code> dentro del clúster
RF-09	Aceptar alias de campos habituales en el cuerpo JSON	Validadores de Pydantic en <code>main.py</code>
RF-10	Normalizar la fecha de nacimiento a ISO YYYY-MM-DD	<code>_normalize_fecha_nacimiento</code>

El identificador lo genera la API (UUID versión 4). El cliente no lo envía en el alta. El precio se guarda como `Decimal` porque DynamoDB no acepta flotantes binarios de Python. La fecha admite YYYY-MM-DD o DD/MM/YYYY y siempre se persiste en ISO.

5.2 Requisitos no funcionales

ID	Requisito	Decisión
RNF-01	Reproducibilidad	CloudFormation y manifiestos en Git; parámetros en <code>vars.yaml</code>
RNF-02	Mínimo costo en dev	Spot, on-demand DynamoDB, un NAT, réplicas bajas
RNF-03	Sin llaves de larga duración en el pod	IRSA
RNF-04	Cómputo de aplicación fuera de Internet	Subnets privadas y NLB interno
RNF-05	Cifrado de secretos de Kubernetes	CMK de KMS en <code>Encryption</code> <code>Config</code> del clúster
RNF-06	Tiempo de arranque conocido en modo <code>ConfigMap</code>	Sondas con espera inicial de 60 s y 90 s
RNF-07	Tope de cómputo elástico	<code>limits.cpu: "4"</code> en el <code>NodePool</code>

ID	Requisito	Decisión
RNF-08	Documentación regenerable	<code>make docs-pdf</code>

5.3 Trazabilidad breve

Cada requisito funcional termina en un método de `k8s/fastapi/main.py` y en un ítem de DynamoDB. Cada requisito de plataforma termina en una plantilla o en un manifiesto. La prueba de humo (`make smoke`) cubre el camino público de listado: espera HTTP 200 en `/stage/productos` y en `/stage/clientes`. No sustituye una suite de contrato; comprueba que el camino API Gateway → NLB → Istio → pod → DynamoDB está vivo.

6. Decisiones de arquitectura

Las decisiones siguientes explican por qué la topología es esta y no una más simple o una más grande.

API Gateway delante de un NLB interno. Un Service LoadBalancer público habría publicado los puertos del ingress en Internet. API Gateway aporta la URL estable, el stage y el punto único donde más adelante se puede colgar autorización. El VPC Link es el mecanismo de REST API para llegar a un balanceador de red que no tiene dirección pública.

Istio como ingress del clúster. El enrutamiento por prefijo (`/productos`, `/clientes`) vive en un VirtualService. Así el Service de Kubernetes no necesita un Ingress clásico ni reglas por ruta en el balanceador. El perfil de Istio se recortó en CPU y memoria para caber en un nodo `t3a.medium` o incluso en un `t3a.small`.

Karpenter además de un node group pequeño. El managed node group existe para que el clúster tenga dónde correr Karpenter, Istio y los addons. Las cargas de la aplicación pueden caer en nodos que Karpenter crea al ver pods Pending. El pool solo admite Spot, arquitectura amd64 y los tipos `t3a.small` y `t3a.medium`.

DynamoDB on-demand y sin índices secundarios. En laboratorio el tráfico es bajo y los listados caben en un `Scan`. No se paga capacidad provisionada ociosa. El costo de esa simplicidad es que un `Scan` lee la tabla completa y la implementación actual no sigue `LastEvaluatedKey`, así que un resultado mayor de 1 MB quedaría truncado.

ConfigMap como modo por defecto de la aplicación. Construir una imagen local no sirve en los nodos EKS: esa imagen no existe en el nodo. Mientras no haya una URI de ECR en `FASTAPI_IMAGE`, `make app` crea un ConfigMap con `main.py` y `requirements.txt` y arranca `python:3.12-slim`, instalando dependencias en el inicio del contenedor.

Stacks separados en lugar de un solo archivo. Cada capa se puede actualizar sin tocar las demás. El stack maestro `template.yaml` existe como opción de stacks anidados, pero exige subir

las plantillas a S3 con `aws cloudformation package`. El camino documentado y usado por `make infra` es el despliegue secuencial de `scripts/deploy.sh`.

API en dos fases. Si `NlbArn` está vacío, la plantilla de API Gateway crea la REST API y omite VPC Link, métodos y stage (`Condition: HasNlb`). Primero se levanta la plataforma, se descubre el NLB y después se despliega la fachada. Así CloudFormation no exige un balanceador que todavía no existe.

7. Arquitectura lógica

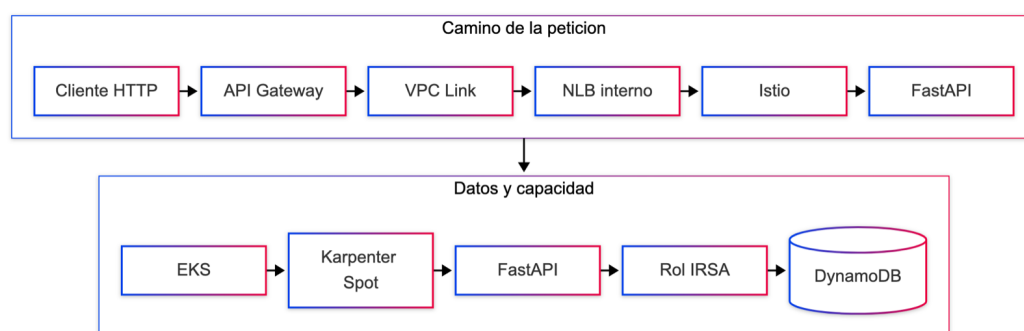


Figura 1: Arquitectura lógica: del cliente a DynamoDB

La figura anterior resume los componentes y no la red física. El cliente HTTP solo conoce API Gateway. A partir de ahí el camino es privado: VPC Link, NLB interno, Istio, el pod FastAPI y DynamoDB. El rol IRSA no transporta la petición; autoriza las llamadas de boto3. Karpenter no está en el camino de datos: observa pods sin nodo y añade capacidad. El plano de control de EKS programa esa capacidad y mantiene el estado del clúster.

Componentes y responsabilidad

Componente	Responsabilidad	Qué no hace
API Gateway	Frontera HTTPS, stage, proxy HTTP	No contiene lógica de negocio
VPC Link	Túnel de la REST API hacia el NLB	No balancea entre pods
NLB interno	Balanceo TCP hacia el ingress	No enruta por ruta HTTP
Istio Gateway	Acepta HTTP en el puerto declarado	No elige la tabla DynamoDB
VirtualService	Prefijos <code>/productos</code> y <code>/clientes</code>	No publica el balanceador
FastAPI	Validación, UUID, CRUD	No abre puertos en la VPC
DynamoDB	Persistencia de ítems	No autentica al usuario final
IRSA	Credencial temporal del pod	No sustituye la autorización de la API

Componente	Responsabilidad	Qué no hace
Karpenter	Nodos Spot bajo demanda	No escala réplicas del Deployment
EKS	Plano de control y nodos	No es la fachada pública

El Deployment de la aplicación declara una réplica en modo ConfigMap (dos si se usa imagen ECR). Karpenter reacciona a la demanda de pods, no al RPS. No hay Horizontal Pod Autoscaler en esta versión. Si hiciera falta más caudal, habría que subir réplicas y, si el nodo no alcanza, Karpenter sumaría una instancia dentro del tope de CPU del pool.

8. Arquitectura de red

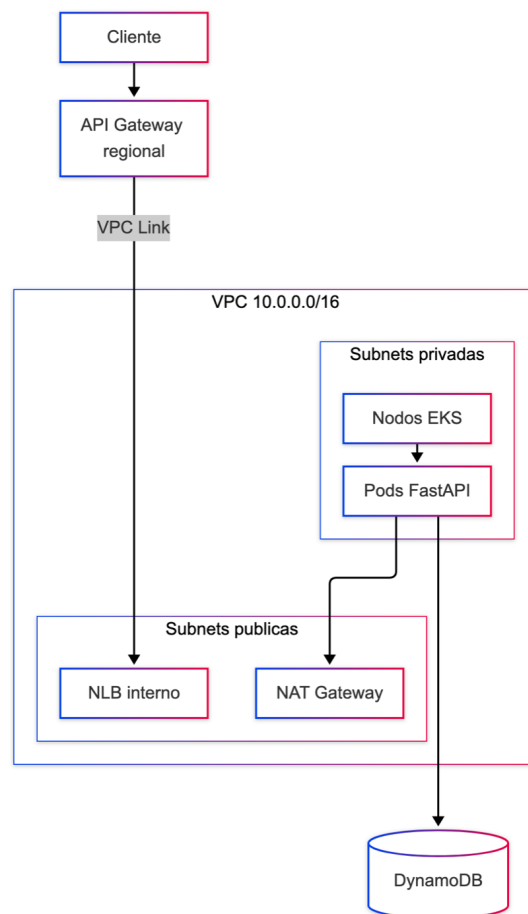


Figura 2: Subnets, NAT y camino del NLB interno

Recurso	Detalle
CIDR de la VPC	10.0.0.0/16, parámetro <code>VpcCidr</code>
Subnets públicas	Dos, una por zona, con IP pública al lanzar
Subnets privadas	Dos, una por zona; aquí viven los workers

Recurso	Detalle
NAT Internet Gateway	Uno, en la subnet pública de la primera zona Asociado a la VPC; lo usan las subnets públicas
EKS	Endpoint público y privado; ENIs del clúster en subnets privadas
NLB	Interno, creado por el Service de Kubernetes

Plantilla: `vpc/template.yaml`.

8.1 Direccionamiento

`Fn::Cidr` parte el bloque de la VPC en cuatro subredes. Con el tercer argumento 8 sobre un bloque /16, cada subred resultante es un /24. El segundo argumento pide cuatro bloques. El orden de `!Select` en la plantilla es el siguiente:

Índice	Nombre lógico	Uso	CIDR si la VPC es 10.0.0.0/16
0	<code>PublicSubnetAz1</code>	NAT y salida pública	10.0.0.0/24
1	<code>PublicSubnetAz2</code>	Segunda zona pública	10.0.1.0/24
2	<code>PrivateSubnetAz1</code>	Workers	10.0.2.0/24
3	<code>PrivateSubnetAz2</code>	Workers	10.0.3.0/24

Cada /24 ofrece 256 direcciones, de las cuales AWS reserva cinco. El espacio alcanza para un nodo bootstrap, unos pocos nodos de Karpenter y los ENI del CNI. No alcanza para una flota grande: el tope del NodePool (`cpu: "4"`) mantiene el laboratorio dentro de ese espacio. Si `VpcCidr` cambia, los cuatro bloques se recalculan solos; no hay CIDR de subnet escritos a mano.

Las zonas salen de `!GetAZs` de la región: índice 0 e índice 1. No se fijan nombres de zona (`us-east-1a`, etc.) para que la plantilla sirva en otra región.

8.2 Tablas de ruta

La tabla pública envía 0.0.0.0/0 al Internet Gateway. La tabla privada envía 0.0.0.0/0 al NAT Gateway. Ambas subnets privadas comparten esa tabla, así que toda salida a Internet (pull de imágenes, llamadas a DynamoDB, STS, a la API de EC2 que hace Karpenter) sale por un único NAT.

DynamoDB se consume por el endpoint público regional, no por un VPC endpoint. Por eso el NAT es obligatorio mientras los nodos estén en subnets privadas. Un gateway endpoint de DynamoDB y un interface endpoint de STS reducirían el tráfico del NAT; no están en la plantilla de esta versión y se listan como mejora.

8.3 Etiquetas que la plataforma necesita

Etiqueta	Dónde	Para qué
<code>kubernetes.io/role/elb=1</code>	Subnets públicas	Balanceadores públicos, si algún día se crean
<code>kubernetes.io/role/internal-elb=1</code>	Subnets privadas	El NLB interno elige estas subnets
<code>kubernetes.io/cluster/{ClusterName}=shared</code>	Subnets privadas	Descubrimiento clásico de EKS
<code>karpenter.sh/discovery={ClusterName}</code>	Subnets privadas	El EC2NodeClass selecciona subnets

Sin `internal-elb`, el Service LoadBalancer interno puede quedar sin dirección. Sin `karpenter.sh/discovery`, Karpenter no encuentra subnets y los pods siguen Pending.

8.4 Superficie de red

El cliente de Internet abre HTTPS contra `execute-api`. No hay ruta pública hacia el puerto 8000 del pod ni hacia el puerto 80 del NLB. El endpoint público del API server de EKS sí existe (`EndpointPublicAccess: true`), protegido por IAM de AWS, no por la API de negocio. Conviene restringir los CIDR de ese endpoint en un entorno que ya no sea un laboratorio; la plantilla actual no añade `PublicAccessCidrs` distintos del valor por defecto.

El grupo de nodos admite todo el tráfico que viene de sí mismo (regla nodo a nodo). El stack `ec2/` añade ingreso TCP 80–8080 desde el grupo de seguridad asociado al camino del NLB, y el grupo de targets abre 80, 8080 y 15021 desde `10.0.0.0/8`. El puerto 15021 es el health de Istio; 8080 es el puerto real del contenedor ingreso; 80 es el puerto del Service.

9. Recorrido de una petición



Figura 3: Puertos y saltos de una llamada HTTPS

El ejemplo es un alta de producto. La misma cadena, con otro prefijo, sirve para clientes.

1. El cliente envía `POST https://{api-id}.execute-api.{region}.amazonaws.com/dev/productos` con un JSON de nombre, precio y stock. TLS termina en API Gateway.
2. El método `ANY` del recurso `/productos` está integrado como `HTTP_PROXY` con `Connection Type: VPC_LINK`. La URI de integración es `http://{NlbDnsName}/productos`. El nombre del stage no forma parte de esa URI.
3. El VPC Link entrega la conexión al NLB interno. El listener efectivo del Service es el puerto 80.
4. El NLB reparte TCP hacia los pods del ingress de Istio. El `targetPort` del Service es 8080, que es donde escucha el contenedor `istio-ingressgateway`.
5. El Gateway `api-mercado-gateway` acepta HTTP en el puerto 80 del plano de Istio (hosts * en este laboratorio) y el VirtualService `fastapi` compara el prefijo.
6. El destino es `fastapi.api-mercado.svc.cluster.local:8000`. kube-proxy o el modo IP del CNI entrega el paquete al pod. Si la inyección de Istio está activa, el sidecar Envoy del pod también participa.
7. Uvicorn invoca `create_producto`. Pydantic valida el cuerpo. La función genera un UUID,

convierte el precio a `Decimal` y hace `PutItem` en la tabla de productos.

8. `boto3` firma la llamada con credenciales temporales del rol `IRSA`. `DynamoDB` responde y `FastAPI` devuelve 201 con el producto, incluido `productoId`.
9. La respuesta vuelve por el mismo camino. `API Gateway` la entrega al cliente.

Una consulta de listado usa `Scan` en lugar de `PutItem`. Un borrado usa `DeleteItem` y responde 204 sin cuerpo. Un identificador ausente responde 404. Un precio no positivo o una fecha ilegible responden 422, que es el código de validación de `FastAPI`.

`GET /health` no está mapeado en `API Gateway`. Lo usan la sonda del `Deployment` y cualquier prueba ejecutada dentro de la red del clúster. La prueba de humo pública pega a `/productos` y `/clientes`, que sí existen en la fachada.

10. Parámetros globales (`vars.yaml`)

Todos los comandos de `Makefile` leen `vars.yaml` a través de `scripts/load-vars.sh`. No hay un segundo archivo de entorno. Los valores de laboratorio que el repositorio trae como referencia son estos:

Clave	Valor de referencia	Efecto
<code>AwsRegion</code>	<code>us-east-1</code>	Región de la CLI y de los pods
<code>AwsProfile</code>	perfil local	Se exporta como <code>AWS_PROFILE</code> si no está vacío
<code>Environment</code>	<code>dev</code>	Sufijo de nombres de stack y de roles
<code>ProjectName</code>	<code>api-mercado</code>	Prefijo de nombres
<code>VpcCidr</code>	<code>10.0.0.0/16</code>	Bloque de la VPC
<code>Kubernetes</code>	<code>1.33</code>	Versión del plano de control
<code>Version</code>		
<code>ClusterName</code>	<code>api-mercado-dev</code>	Nombre EKS y valor de las etiquetas de descubrimiento
<code>Bootstrap</code>	<code>t3a.medium</code>	Tipo del node group inicial
<code>InstanceTypes</code>		
<code>Bootstrap</code>	<code>1</code>	Nodos deseados al crear
<code>Desired</code>		
<code>Capacity</code>		

Clave	Valor de referencia	Efecto
BootstrapMin Size / MaxSize	1 / 2	Rango del node group
Bootstrap CapacityType	SPOT	El bootstrap también puede ser Spot
ProductosTable Name	dev-productos	Nombre físico de la tabla
ClientesTable Name	dev-clientes	Nombre físico de la tabla
ApiStageName	dev	Stage de API Gateway
Karpenter InstanceTypes	t3a.small, t3a.medium	Lista blanca del NodePool
KarpenterMax Cpu	"4"	Tope agregado del pool
NlbArn, NlbDns Name	vacíos hasta descubrir	Los rellena make discover-nlb
KarpenterChart Version	0.37.0	Chart OCI de Karpenter
IstioVersion	1.22.2	Versión esperada de istioctl
Templates Bucket	vacío	Solo si se usa el stack maestro

El node group de la plantilla EKS tiene por defecto `t3a.small`. `vars.yaml` lo sube a `t3a.medium` porque Istio, Karpenter y el addon de DNS juntos se quedan cortos de memoria en 2 GiB. El script pasa `BootstrapInstanceTypes` al parámetro `BootstrapInstanceType`.

`NlbArn` y `NlbDnsName` se escriben en el mismo YAML después de crear el Service. No deben documentarse con el valor de una cuenta concreta.

11. Stacks CloudFormation

Orden de `make infra`, que invoca `scripts/deploy.sh`:

Orden	Stack	Carpeta	Depende de
1	{Project}- {Env}-vpc	vpc/	Nada

Orden	Stack	Carpeta	Depende de
2	{Project}-{Env}-kms	kms/	Nada en recursos; se despliega segundo por orden del script
3	{Project}-{Env}-dynamodb	dynamodb/	Nada en recursos
4	{Project}-{Env}-iam	iam/	ARN de las dos tablas
5	{Project}-{Env}-eks	eks/	VPC, roles, KMS, policies, cola SQS
6	{Project}-{Env}-ec2	ec2/	VPC, nombre del clúster, SG de nodos
7	{Project}-{Env}-api	api_gateway/	NLB ya existente; no forma parte de <code>make infra</code>

`make infra` se detiene en el stack `ec2`. El stack de API Gateway corre en `make api`, cuando el NLB ya tiene ARN y DNS. Cada `aws cloudformation deploy` usa `CAPABILITY_NAMED_IAM` porque hay roles con nombre explícito, y `--no-fail-on-empty-changeset` para que repetir el comando sea idempotente.

El prefijo de stack es `{ProjectName}-{Environment}`. Con los valores de referencia, el primero se llama `api-mercado-dev-vpc`.

11.1 Encadenamiento de salidas

`deploy.sh` no usa `Fn::ImportValue` entre despliegues sueltos. Lee la salida con `scripts/stack-output.sh` y la inyecta como parámetro del siguiente stack. El acoplamiento queda en el script, no en exportaciones cruzadas obligatorias. Las plantillas sí exportan varios valores (`Export.Name`) por si otra herramienta quiere importarlos.

La cadena real hacia EKS es:

- De DynamoDB: `ProductosTableArn`, `ClientesTableArn` → parámetros del stack IAM.
- De IAM: ARN del rol del clúster, ARN del rol de nodo, ARN de la policy de Karpenter, ARN de la policy de FastAPI, ARN de la cola de interrupciones.
- De KMS: `EksSecretsKeyArn`.
- De VPC: `VpcId`, `PrivateSubnetIds`, `PublicSubnetIds`.
- De EKS hacia EC2: `NodeSecurityGroupId`.

11.2 Stack maestro opcional

`template.yaml` en la raíz anida las mismas plantillas con `AWS::CloudFormation::Stack` y URL en S3 (`TemplatesBucket` y `TemplatesPrefix`). El anidado incluye VPC, KMS, DynamoDB, IAM, EKS y EC2. No sustituye el descubrimiento del NLB: los parámetros `NlbArn` y `NlbDnsName` siguen

llegando desde fuera. El camino soportado por el Makefile es el secuencial. El anidado exige `aws cloudformation package` porque CloudFormation no lee carpetas locales en un stack anidado.

12. Amazon VPC

Archivo: `vpc/template.yaml` **Propósito:** aislar el clúster y definir por dónde sale a Internet y dónde puede nacer un balanceador interno.

Recurso	Función
<code>AWS::EC2::VPC</code>	DNS hostnames y DNS support activos; el CIDR es el parámetro
<code>AWS::EC2::InternetGateway</code> y attachment	Salida y entrada de las subnets públicas
<code>AWS::EC2::Subnet</code> (cuatro)	Dos públicas y dos privadas, en dos zonas
<code>AWS::EC2::EIP</code> y <code>AWS::EC2::NatGateway</code>	Una IP elástica y un NAT en la subnet pública de la zona 1
<code>AWS::EC2::RouteTable</code> y rutas	Pública hacia el IGW; privada hacia el NAT
Asociaciones	Cada subnet queda unida a su tabla

`MapPublicIpOnLaunch` es verdadero solo en las públicas. El NAT depende del attachment del Internet Gateway: sin ruta pública, la IP elástica no se puede usar.

Salidas: `VpcId`, listas separadas por coma de subnets públicas y privadas, y el identificador de cada subnet. `deploy.sh` pasa las listas al parámetro `CommaDelimitedList` del stack EKS.

Fallo típico: cambiar `VpcCidr` después de crear la VPC reemplaza la VPC y, en cascada, casi todo lo que guarda su identificador. En un laboratorio se destruye el entorno y se vuelve a crear; no se edita el CIDR en caliente.

13. AWS KMS

Archivo: `kms/template.yaml` **Propósito:** cifrado en reposo de los secretos de Kubernetes guardados en etcd del plano de control.

Recurso	Configuración
<code>AWS::KMS::Key</code>	CMK dedicada, <code>EnableKeyRotation: true</code>
Policy <code>EnableRoot</code>	La raíz de la cuenta puede administrar la clave
Policy <code>AllowEksService</code>	<code>eks.amazonaws.com</code> puede cifrar, descifrar, describir, generar data keys y re-cifrar
<code>AWS::KMS::Alias</code>	<code>alias/{Project}-{Environment}-eks</code>

La salida `EksSecretsKeyArn` entra al clúster como `EncryptionConfig.Provider.KeyArn` sobre el recurso `secrets`. No cifra volúmenes de nodo ni las tablas DynamoDB. DynamoDB cifra en reposo con la clave propiedad de AWS; esta versión no define una CMK para las tablas.

La rotación automática genera material nuevo cada año. El alias no cambia, así que el clúster sigue apuntando al mismo ARN. Borrar la clave con secretos todavía cifrados deja el clúster sin poder leerlos: la clave no se debe destruir mientras el clúster exista.

14. Amazon DynamoDB

Archivo: `dynamodb/template.yaml` **Propósito:** persistencia del dominio, sin servidor que administrar.

Tabla	Clave de partición	Modo de capacidad	Nombre de referencia
Productos	<code>productoId</code> (String)	<code>PAY_PER_REQUEST</code>	<code>dev-productos</code>
Clientes	<code>clienteId</code> (String)	<code>PAY_PER_REQUEST</code>	<code>dev-clientes</code>

No hay claves de ordenación ni índices secundarios. `GetItem`, `PutItem` y `DeleteItem` usan la clave de partición. Los listados usan `Scan`. Las etiquetas `Project` y `Environment` marcan el recurso para costos y para búsqueda en la consola.

Salidas exportadas: nombre y ARN de cada tabla. El ARN alimenta la policy IAM. El nombre viaja al pod como variable de entorno (`PRODUCTOS_TABLE`, `CLIENTES_TABLE`), no como ARN: `boto3` abre la tabla por nombre y región.

14.1 Ítems de producto

Atributo	Tipo en DynamoDB	Origen
<code>productoId</code>	String	UUID generado en la API
<code>nombre</code>	String	Cuerpo JSON, longitud mínima 1
<code>precio</code>	Number	<code>Decimal</code> construido desde el valor validado
<code>stock</code>	Number	Entero mayor o igual que cero

La API también tolera, al leer, atributos legacy o alias (`producto_id`, `Nombre`, `name`, `Precio`, `price`, `Stock`, `cantidad`, `quantity`). Al escribir, normaliza a `productoId`, `nombre`, `precio` y `stock`. Si un ítem antiguo no trae los cuatro datos, la respuesta es 500 con el detalle de registro incompleto, en lugar de un 200 a medias.

14.2 Ítems de cliente

Atributo	Tipo	Origen
clienteId	String	UUID
nombre	String	Cuerpo, longitud mínima 1
fechaNacimiento	String	ISO YYYY-MM-DD

Si el ítem guardado todavía trae `fecha_nacimiento`, la lectura lo renombra y lo normaliza. Se acepta entrada DD/MM/YYYY (día y mes de uno o dos dígitos). Cualquier otro formato falla la validación.

14.3 Consecuencias del modelo

- Un `Scan` sin paginación en código devuelve como máximo el primer megabyte. Para un laboratorio de decenas de ítems es suficiente. Para un catálogo real hay que iterar `LastEvaluatedKey` o cambiar el patrón de acceso.
- No se puede consultar “productos por nombre” sin leer la tabla. Un GSI sobre `nombre` sería el cambio natural.
- On-demand absorbe un pico de la prueba de humo sin capacidad provisionada. El precio por millón de lecturas y escrituras se revisa en la lista de precios de la región; no se fija en este documento porque cambia.

15. AWS IAM

Archivo: `iam/template.yaml` **Propósito:** identidades del plano de control, de las instancias y las políticas que luego se adjuntan a roles IRSA.

Rol o policy	Quién lo asume	Políticas administradas o documento
<code>EksClusterRole</code>	<code>eks.amazonaws.com</code>	<code>AmazonEKSClusterPolicy</code> , <code>AmazonEKSVPCKResourceController</code>
<code>EksNodeRole</code>	<code>ec2.amazonaws.com</code>	Worker, CNI, ECR de solo lectura, SSM core
<code>KarpenterNodeRole</code>	<code>ec2.amazonaws.com</code>	Las mismas cuatro políticas de nodo
<code>Instance profile Karpenter</code>	Instancias que lanza Karpenter	Apunta a <code>KarpenterNodeRole</code>
<code>Karpenter ControllerPolicy</code>	Se adjunta en el stack EKS al rol IRSA	EC2, <code>PassRole</code> , instance profiles, <code>DescribeCluster</code> , SSM público, pricing, SQS
<code>FastApiDynamoDb Policy</code>	Se adjunta en el stack EKS al rol del pod	CRUD y <code>Scan</code> sobre los dos ARN de tabla
<code>Cola SQS</code>	Karpenter	Retención 300 s, cifrado administrado por SQS

Los roles IRSA no nacen aquí. Nacen en `eks/template.yaml` porque el documento de confianza necesita el ARN del proveedor OIDC, y ese proveedor solo existe cuando el clúster ya está creado.

15.1 Policy del controlador Karpenter

El controlador necesita crear y terminar instancias, leer la oferta de tipos y de precios Spot, etiquetar, pasar el rol de nodo y manipular instance profiles. El documento lo concede sobre `Resource: "*" en esas acciones de EC2 e IAM de perfiles`. Es el patrón que el proyecto Karpenter documenta para poder lanzar en cualquier subnet etiquetada; no es una policy mínima al recurso. `iam:PassRole` sí está limitado al ARN de `KarpenterNodeRole`.

`eks:DescribeCluster` permite al controlador leer el endpoint y el certificado. `pricing:GetProducts` alimenta la decisión de precio. La cola de interrupciones se nombra `{Project}-{Environment}-karpenter`. La retención de 300 segundos alcanza para que el controlador vea el aviso de Spot y drene el nodo; no es una cola de trabajo de la aplicación.

15.2 Policy de FastAPI

Acciones permitidas, solo sobre los ARN de las dos tablas: `GetItem, PutItem, UpdateItem, DeleteItem, Query, Scan, BatchGetItem, BatchWriteItem, DescribeTable`. No hay `dynamodb:* sobre *`. El código actual usa `Get, Put, Delete y Scan`. `Query` y los `batch` quedan autorizados por si el código crece, sin abrir otras tablas de la cuenta.

16. Amazon EKS

Archivo: `eks/template.yaml` **Propósito:** plano de control, nodo mínimo, OIDC e IRSA.

Recurso	Detalle
<code>NodeSecurityGroup</code>	Tráfico de todos los protocolos desde el mismo grupo; etiqueta <code>kubernetes.io/cluster/{nombre}=owned</code>
<code>AWS::EKS::Cluster</code>	Versión <code>KubernetesVersion</code> ; subnets privadas; endpoint público y privado
<code>EncryptionConfig</code>	Recurso <code>secrets</code> con la CMK
<code>AWS::IAM::OIDCProvider</code>	Audiencia <code>sts.amazonaws.com</code> ; el emisor es el del clúster
<code>KarpenterControllerRole</code>	Confianza federada al SA <code>kube-system:karpenter</code>
<code>FastApiPodRole</code>	Confianza federada al SA <code>api-mercado:fastapi</code>

Recurso	Detalle
BootstrapNodeGroup	Nombre con sufijo <code>bootstrap-al2023</code> , AMI <code>AL2023_x86_64_STANDARD</code> , etiqueta <code>role=bootstrap</code>
Addons	<code>vpc-cni</code> , <code>kube-proxy</code> , <code>coredns</code> con <code>ResolveConflicts: OVERWRITE</code>
CoreDNS	Depende del node group, para no quedar Pending sin ningún nodo

16.1 Nodo bootstrap y Amazon Linux 2023

El grupo se llama `{Project}-{Environment}-bootstrap-al2023` a propósito. EKS rechaza con 409 un alta de node group si el nombre ya existe y CloudFormation intenta crear el nuevo antes de borrar el viejo. Al cambiar el `AmiType` de AL2 a AL2023 el reemplazo necesita un nombre distinto. El comentario de la plantilla deja esa razón escrita junto al recurso.

`CapacityType` admite `SPOT` u `ON_DEMAND`. El laboratorio usa Spot también en el bootstrap. Si Spot no tiene capacidad en las dos zonas, el node group puede tardar o fallar el despliegue: la mitigación operativa es cambiar `BootstrapCapacityType` a `ON_DEMAND` el tiempo necesario para instalar la plataforma y volver a Spot después, o ampliar tipos de instancia.

El rango por defecto de la plantilla es mínimo 1, máximo 2, deseado 1, tipo `t3a.small`. `vars.yaml` solo cambia el tipo a `t3a.medium`. Un `t3a.medium` tiene 2 vCPU y 4 GiB. Ahí conviven el controlador de Karpenter (una réplica), `istiod` recortado, el `ingress` recortado y CoreDNS. La aplicación puede caer en ese mismo nodo si cabe, o en un nodo nuevo de Karpenter.

16.2 IRSA

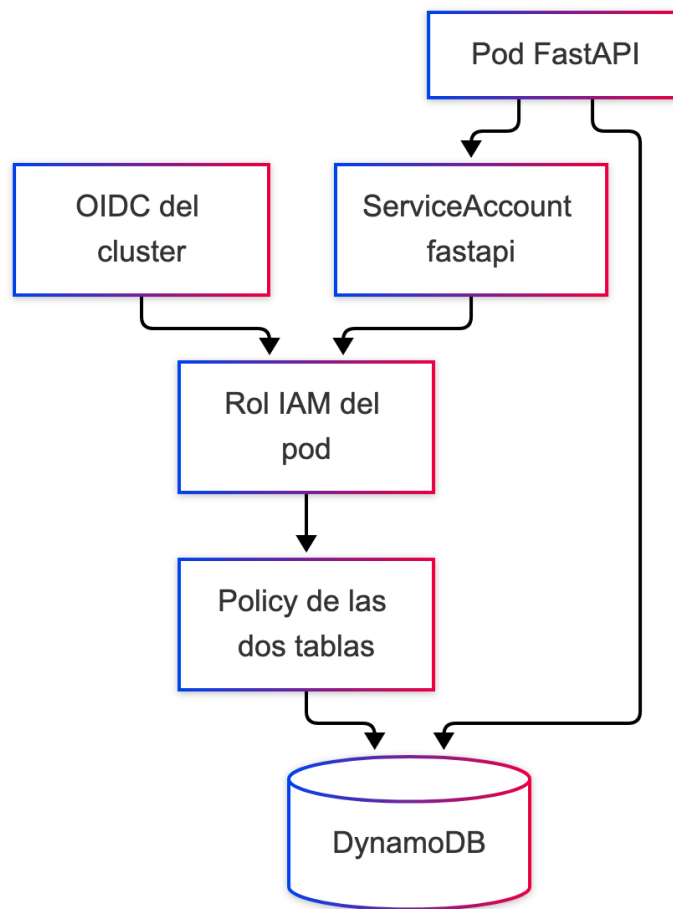


Figura 4: Identidad del pod frente a DynamoDB

El proveedor OIDC publica el emisor del clúster. Cada rol de workload exige en la condición **String Equals**:

- `{emisor}:aud = sts.amazonaws.com`
- `{emisor}:sub` = el nombre exacto de la cuenta de servicio

Para FastAPI el subject es `system:serviceaccount:api-mercado:fastapi`. Para Karpenter es `system:serviceaccount:kube-system:karpenter`. Otro namespace u otro nombre de ServiceAccount no puede asumir el rol aunque conozca el ARN. El pod no lleva `AWS_ACCESS_KEY_ID`. El webhook de EKS inyecta las variables de token web y el SDK las cambia por credenciales temporales.

La huella del proveedor OIDC está fijada en la plantilla al valor histórico del certificado raíz que IAM documentó para estos proveedores (`9e99a48a9960b14926bb7f3b02e22da0ecd2a979`). Si AWS rota la cadena y el alta del proveedor falla, ese campo es el que hay que actualizar según la documentación vigente de EKS, no un secreto del proyecto.

16.3 Addons

`vpc-cni` asigna direcciones de la subnet del nodo a los pods. `kube-proxy` programa las reglas de

Service. `coredns` resuelve nombres internos, entre ellos el host del `VirtualService`. Los tres usan la versión por defecto compatible con la versión del clúster (`ResolveConflicts: OVERWRITE` para que un cambio de configuración de la plantilla gane sobre ajustes manuales). No se instala el AWS Load Balancer Controller como chart. El NLB nace de las anotaciones del Service, que atiende el controlador de nube integrado en EKS.

17. Stack auxiliar EC2 y SSM

Archivo: `ec2/template.yaml` **Propósito:** preparar el camino de red hacia el ingress y dejar datos operativos en Parameter Store. No lanza instancias de la aplicación.

Recurso	Detalle
<code>NlbTargetSecurityGroup</code>	Ingreso TCP 80, 8080 y 15021 desde 10.0.0.0/8; egreso libre
<code>NodeToNlbIngress</code>	En el SG de nodos, TCP 80–8080 desde el SG anterior
<code>SSM cluster-name</code>	<code>/{{Project}}/{{Environment}}/cluster-name</code>
<code>SSM node-security-group-id</code>	El SG de nodos del stack EKS
<code>SSM karpenter-discovery</code>	El nombre del clúster, que es el valor de la etiqueta

El NLB y sus grupos de seguridad efectivos los crea el controlador de nube al ver el Service. Este stack deja un grupo coherente con los puertos de Istio y una regla para que el nodo acepte ese tráfico. Los parámetros SSM permiten a un operador o a otro stack leer el nombre del clúster y el SG sin volver a consultar CloudFormation.

18. Amazon API Gateway

Archivo: `api_gateway/template.yaml` **Propósito:** fachada HTTPS regional hacia el backend privado.

Recurso	Detalle
<code>AWS::ApiGateway::RestApi</code>	Endpoint REGIONAL, nombre <code>{{Project}}-{{Environment}}-api</code>
<code>AWS::ApiGateway::VpcLink</code>	Solo si hay NLB; <code>TargetArns</code> es el ARN del balanceador
Recursos	<code>/productos</code> , <code>/productos/{{proxy+}}</code> , <code>/clientes</code> , <code>/clientes/{{proxy+}}</code>

Recurso	Detalle
Métodos	ANY, AuthorizationType: NONE, condición HasNlb
Integración	HTTP_PROXY, VPC_LINK, Passthrough Behavior: WHEN_NO_MATCH
Deployment y Stage	El stage usa ApiStageName

URI de integración:

- Colección de productos: `http://{NlbDnsName}/productos`
- Elemento o subruta: `http://{NlbDnsName}/productos/{proxy}`
- Clientes, igual con el prefijo `/clientes`

El método proxy marca `method.request.path.proxy` como obligatorio y lo copia a `integration.request.path.proxy`. Así `GET /dev/productos/{uuid}` llega al pod como `GET /productos/{uuid}`.

Condición HasNlb. `NlbArn` vacío es falso. En ese caso existen la REST API y los recursos de ruta, pero no el VPC Link, ni los métodos, ni el deployment, ni el stage. Sirve para un primer pase de la plantilla. El pase útil es el de `make api`, con ARN y DNS ya escritos.

Salidas: `RestApiId`, `VpcLinkId` e `InvokeUrl`. La URL de invocación es:

`https://{rest-api-id}.execute-api.{region}.amazonaws.com/{stage}`

`make smoke` lee esa salida. No incluye `/productos`: el script lo concatena.

Hueco de seguridad consciente: sin autorizador, cualquier cliente que conozca la URL puede leer y borrar datos del laboratorio. No hay API key, no hay WAF y no hay cuota en la plantilla. Antes de cargar datos reales hay que cerrar el método con Cognito, IAM o un autorizador Lambda, y restringir el stage.

19. Plataforma Kubernetes

Pieza	Ubicación	Notas de laboratorio
Karpenter	Chart OCI 0.37.x, una réplica	CRD v1beta1, no v1
EC2NodeClass	<code>k8s/karpenter/ec2nodeclass.yaml</code>	<code>amiFamily</code> : AL2023
NodePool	<code>k8s/karpenter/nodepool.yaml</code>	Spot, <code>t3a.small</code> y <code>t3a.medium</code>
Istio	<code>k8s/istio/values-minimal.yaml</code>	IstioOperator con requests bajos

Pieza	Ubicación	Notas de laboratorio
Gateway y NLB	k8s/istio/gateway-internal-nlb.yaml	Service interno más Gateway
Aplicación	k8s/app/	Namespace, SA, Deployment, Service, VirtualService
Código	k8s/fastapi/main.py	FastAPI y boto3

`make platform` instala el chart, aplica Istio, el Gateway, y renderiza el `EC2NodeClass` con `envsubst` antes de aplicarlo. El `NodePool` no lleva variables y se aplica tal cual. La aplicación va en un paso distinto (`make app`) para poder repetir el despliegue del código sin reinstalar la malla.

20. Karpenter

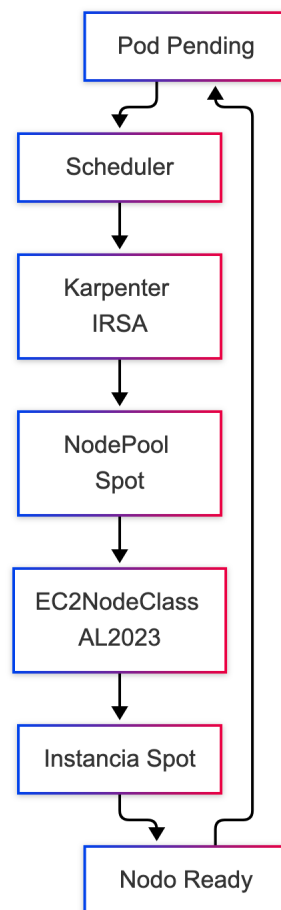


Figura 5: De un pod Pending a un nodo Spot

Karpenter observa pods que el scheduler no puede colocar y lanza una instancia que cumpla el `NodePool`. No es el `Cluster Autoscaler` de un `node group`: el `node group bootstrap` no crece para la aplicación; crece como máximo a dos nodos por su propio `ScalingConfig`. La elasticidad de cargas está en el pool `spot-workloads`.

20.1 Instalación con Helm

`deploy.sh` instala el chart `oci://public.ecr.aws/karpenter/karpenter` en `kube-system`, versión `KarpenterChartVersion`. Fija `replicas=1` porque el anti-affinity del chart con dos réplicas no cabe en un solo nodo bootstrap y el release queda `pending`. Si el release ya está en `pending-install`, `pending-upgrade` o `pending-rollback`, el script lo desinstala y lo vuelve a instalar.

Valores que se pasan:

- `settings.clusterName` = nombre del clúster
- `settings.interruptionQueue` = nombre de la cola SQS
- Anotación `eks.amazonaws.com/role-arn` en el ServiceAccount del controlador, con el ARN salido del stack EKS

El timeout de espera es 15 minutos.

20.2 EC2NodeClass

Campo	Significado
<code>apiVersion</code>	<code>karpenter.k8s.aws/v1beta1</code> (el chart 0.37 no sirve CRD v1)
<code>amiFamily: AL2023</code>	AMI de EKS sobre Amazon Linux 2023
<code>role</code>	Nombre del rol de nodo Karpenter, sustituido al renderizar
<code>subnetSelectorTerms</code>	Tag <code>karpenter.sh/discovery</code> igual al nombre del clúster
<code>securityGroupSelectorTerms</code>	El mismo tag, más el id explícito del SG de nodos
<code>tags</code>	<code>Project: api-mercado</code> y el tag de descubrimiento

`scripts/render-k8s.sh` sustituye `KARPENTER_NODE_ROLE_NAME`, `CLUSTER_NAME` y `NODE_SECURITY_GROUP_ID`. El resultado queda en `.rendered/k8s/`, que no se versiona. Si el stack EKS no existe, el script termina: sin ARN del rol de FastAPI no tiene sentido aplicar manifiestos.

20.3 NodePool

Campo	Significado
Nombre	<code>spot-workloads</code>
<code>nodeClassRef</code>	<code>EC2NodeClass default</code>
Capacidad	Solo <code>spot</code>
Arquitectura	<code>amd64</code>
Tipos	<code>t3a.small</code> , <code>t3a.medium</code>
<code>limits.cpu</code>	<code>"4"</code>
Consolidación	<code>WhenUnderutilized</code>

`t3a.small` y `t3a.medium` aportan 2 vCPU cada uno. Un tope de 4 vCPU admite del orden de dos nodos de Karpenter además del bootstrap. `t3a.small` tiene 2 GiB de RAM: sirve para un

pod ligero, no para reinstalar Istio ahí. `t3a.medium` tiene 4 GiB y es la opción holgada del mismo tamaño de CPU.

`WhenUnderutilized` permite a Karpenter mover cargas y apagar un nodo de más. En Spot eso también reduce la ventana en la que se paga una instancia vacía. Una interrupción de Spot llega por la cola SQS; el controlador debe drenar el nodo antes de que AWS lo retire. Con una sola réplica de FastAPI, ese drenado se nota como un corte breve. Es coherente con el alcance de laboratorio y no lo es con un acuerdo de disponibilidad.

21. Istio

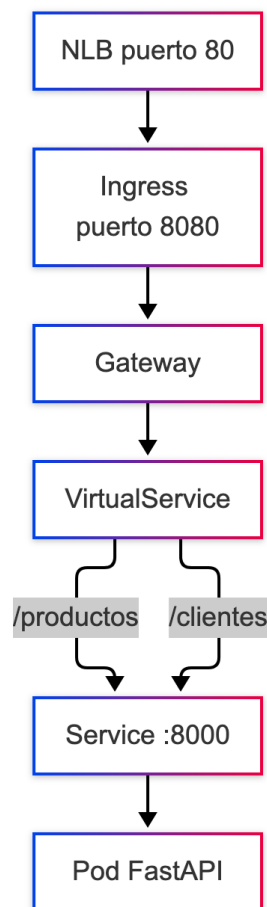


Figura 6: Del NLB al Service de FastAPI

Istio aporta el ingress del clúster y el enrutamiento por prefijo. La inyección de sidecar está habilitada en el namespace de la aplicación (`istio-injection: enabled`). El sidecar suma memoria al pod; por eso los límites del contenedor de la API no son todo el presupuesto del nodo.

21.1 IstioOperator

Archivo `k8s/istio/values-minimal.yaml`. Perfil `default` (istiod más el ingress gateway). Los access logs van a stdout.

Componente	Requests	Limits
pilot (istiod)	100m CPU, 256 MiB	500m CPU, 512 MiB
ingress gateway	50m CPU, 128 MiB	200m CPU, 256 MiB

Esos techos existen para que la malla arranque en el nodo bootstrap. Un perfil por defecto sin recortar pide más RAM de la que deja libre un `t3a.small` y a veces un `t3a.medium` si Karpenter y CoreDNS ya están colocados. `istioctl install -y -f` aplica este operador. La versión de `istioctl` en la máquina debe ser compatible con `IstioVersion` (referencia 1.22.2). El script no descarga Istio: si `istioctl` no está en el PATH, `make platform` falla con un mensaje explícito.

21.2 Service del NLB y Gateway

Archivo `k8s/istio/gateway-internal-nlb.yaml`.

Anotación o campo	Efecto
<code>aws-load-balancer-type: nlb</code>	Balanceador de red, no un CLB clásico
<code>aws-load-balancer-scheme: internal</code>	Sin dirección pública
<code>cross-zone-load-balancing-enabled: true</code>	Reparte entre zonas
Puerto 80, <code>targetPort 8080</code>	El cliente del VPC Link habla por 80; el contenedor escucha en 8080
Tipo <code>LoadBalancer</code>	Pide al controlador de nube que cree el balanceador
Selector	Pods <code>app=istio-ingressgateway</code> e <code>istio=ingressgateway</code>

El Gateway `api-mercado-gateway` vive en `istio-system`, selecciona esos mismos pods y abre un servidor HTTP en el puerto 80 para el host `*`. No hay TLS dentro del clúster: el cifrado hacia el cliente termina en API Gateway. El tramo VPC Link → NLB va en HTTP dentro de la VPC. Es aceptable en este diseño porque el tramo no sale a Internet. Quien necesite cifrado extremo a extremo hasta el pod tendría que añadir un certificado en el Gateway y cambiar la integración; no está hecho.

21.3 VirtualService

Archivo `k8s/app/virtualservice.yaml`, namespace `api-mercado`.

- `gateways: istio-system/api-mercado-gateway`
- Host `*`

- Prefijo `/productos` → `fastapi.api-mercado.svc.cluster.local:8000`
- Prefijo `/clientes` → el mismo Service y puerto

No hay ruta para `/health` en este Gateway. Una petición pública a `/stage/health` no tiene método en API Gateway, así que ni siquiera llega a Istio. Dentro del clúster, el Service sí responde `/health` porque FastAPI lo publica y las sondas hablan directo al contenedor, sin pasar por el VirtualService.

El orden de las reglas no importa aquí: los prefijos no se solapan. Una ruta nueva (por ejemplo `/pedidos`) exigiría otro bloque `http` y, en la fachada, otro recurso de API Gateway. No basta con cambiar solo un lado.

22. Microservicio FastAPI

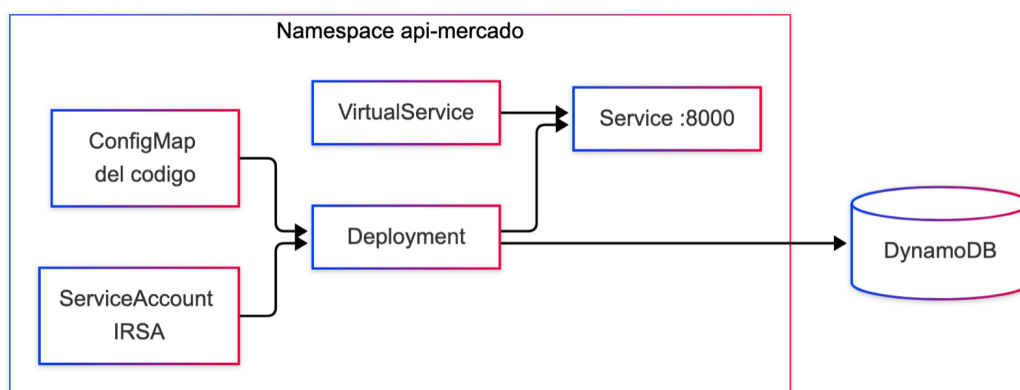


Figura 7: Objetos del namespace api-mercado

22.1 Namespace

`k8s/app/namespace.yaml` crea `api-mercado` con la etiqueta `istio-injection: enabled`. Los pods que nazcan después reciben el sidecar. Un pod ya creado no se inyecta solo: hay que volver a crearlo (`rollout restart` después de activar la etiqueta).

22.2 ServiceAccount

`k8s/app/serviceaccount.yaml` anota `eks.amazonaws.com/role-arn` con el ARN que `render-k8s.sh` lee de la salida `FastApiPodRoleArn`. El nombre de la cuenta es `fastapi`, en el namespace `api-mercado`. Tiene que coincidir carácter por carácter con la condición del rol. El Deployment referencia `serviceAccountName: fastapi`.

22.3 Service

`k8s/app/service.yaml` es un ClusterIP. Puerto 8000, `targetPort` llamado `http`, selector `app: fastapi`. No tiene tipo `LoadBalancer`: no debe crear otro balanceador. El único Service que crea balanceador es el del ingress.

22.4 Dos formas de desplegar

Modo ConfigMap (el predeterminado). Se usa si `FASTAPI_DEPLOY=configmap` o si el modo es `auto` y no hay una imagen ECR distinta del marcador local `api-mercado-fastapi:latest`. El script crea el ConfigMap `fastapi-source` con `main.py` y `requirements.txt` y aplica `deployment-fastapi-configmap.yaml`.

Aspecto	Valor
Imagen	<code>python:3.12-slim</code>
Comando	<code>pip install</code> sin caché y luego <code>uvicorn main:app --host 0.0.0.0 --port 8000</code>
Código	ConfigMap montado en <code>/app</code>
Réplicas	1
Estrategia	<code>maxSurge: 0, maxUnavailable: 1</code>
Variables	<code>AWS_REGION, PRODUCTOS_TABLE, CLIENTES_TABLE</code>
Readiness	<code>GET /health</code> , espera inicial 60 s, cada 10 s, 6 fallos
Liveness	<code>GET /health</code> , espera inicial 90 s, cada 20 s
Recursos	Request 100m / 256 MiB; límite 500m / 512 MiB

`maxSurge: 0` evita pedir un segundo pod durante el rolling update. En un nodo justo de memoria, el segundo pod más el sidecar no entraría y el rollout se quedaría colgado. El costo es una ventana sin réplica lista. La espera de 60 segundos cubre el `pip install` de FastAPI, `uvicorn`, `boto3` y `pydantic`. Si la imagen base todavía no está en el nodo, hay que sumar el pull a través del NAT: la sonda puede fallar y Kubernetes la reintenta hasta el umbral.

Modo imagen. `deployment-fastapi.yaml` usa `${FASTAPI_IMAGE}`, dos réplicas, readiness a los 5 segundos y límites más bajos (128–256 MiB) porque ya no instala paquetes al arrancar. `FASTAPI_DEPLOY=docker` intenta un `docker build` local y avisa de que esa etiqueta no existe en los nodos: hay que subirla a ECR y poner la URI en `FASTAPI_IMAGE`. El repositorio no incluye el ciclo de `docker push`.

Después de aplicar Deployment, Service y VirtualService, el script vuelve a aplicar el Gateway del NLB y espera el rollout como máximo 180 segundos.

22.5 Dependencias

`k8s/fastapi/requirements.txt` fija:

Paquete	Versión
fastapi	0.115.0
uvicorn[standard]	0.30.6
boto3	1.35.16
pydantic	2.9.2

Fijar versiones evita que un `pip install` de mañana cambie el contrato de validación. Subirlas es un cambio deliberado del archivo, no un efecto lateral del despliegue.

22.6 Lógica de aplicación

El proceso crea el recurso `boto3` al importar el módulo, con `AWS_REGION` (defecto `us-east-1`) y los nombres de tabla (defecto `dev-productos` y `dev-clientes`). En el clúster esos defectos no se usan: el manifiesto siempre inyecta las variables renderizadas. Los defectos sirven para una prueba local contra tablas de la misma cuenta, si el desarrollador tiene credenciales en su sesión.

`ProductoIn` exige `nombre` no vacío, `precio` mayor que cero y `stock` mayor o igual que cero. Acepta alias en español y en inglés, ignora campos extra y convierte precio o stock si llegan como texto ("`1,85`" pasa a `1.85`). `ClienteIn` exige nombre y fecha, y normaliza el calendario antes de guardar.

Los listados mapean cada ítem por el DTO de salida. Un ítem corrupto aborta el listado entero con 500. Es estricto a propósito: preferible un error visible a devolver un arreglo con huecos. Create responde 201. Delete responde 204. Get y update responden 404 si `GetItem` no trae ítem. Update reescribe el ítem completo con `PutItem`; no hace `UpdateExpression` parcial. Un campo omitido no se conserva: el cuerpo debe traer el recurso entero.

23. Contrato de la API

La URL pública es `{InvokeUrl}/{ruta}`. `InvokeUrl` ya incluye el stage. Dentro del pod la ruta no lleva stage.

23.1 Productos

Método	Ruta pública	Éxito	Cuerpo
GET	<code>/productos</code>	200	<code>{ "items": [Producto] }</code>
POST	<code>/productos</code>	201	Producto creado
GET	<code>/productos/{id}</code>	200	Producto
PUT	<code>/productos/{id}</code>	200	Producto actualizado
DELETE	<code>/productos/{id}</code>	204	Vacío

Ejemplo de alta:

```
{
  "nombre": "Leche entera 1L",
  "precio": 1.85,
  "stock": 40
}
```

Respuesta 201:

```
{
  "productoId": "3f1c0a2e-7b64-4d1a-9c20-6e5b8a1d4f77",
  "nombre": "Leche entera 1L",
  "precio": 1.85,
  "stock": 40
}
```

El UUID del ejemplo es ilustrativo. La API genera otro en cada alta. Precio 0 o negativo, stock negativo o nombre vacío producen 422. Un id desconocido produce 404 con `{"detail": "Producto no encontrado"}`.

Consulta con la URL de invocación (el host real sale del stack):

```
curl -sS -X POST "$INVOKE/productos" \
  -H 'content-type: application/json' \
  -d '{"nombre": "Leche entera 1L", "precio": 1.85, "stock": 40}'
```

23.2 Clientes

Método	Ruta pública	Éxito
GET	/clientes	200 y { "items": [...] }
POST	/clientes	201
GET	/clientes/{id}	200
PUT	/clientes/{id}	200
DELETE	/clientes/{id}	204

Ejemplo de alta. Las dos fechas son equivalentes; se guarda la forma ISO.

```
{ "nombre": "Ana Ruiz", "fechaNacimiento": "1994-03-12" }
{ "nombre": "Ana Ruiz", "fechaNacimiento": "12/03/1994" }
```

Respuesta:

```
{
  "clienteId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "nombre": "Ana Ruiz",
}
```

```
"fechaNacimiento": "1994-03-12"
}
```

23.3 Health

GET /health en el pod responde {"status":"ok"} sin tocar DynamoDB. No demuestra que las tablas existan ni que el rol IAM funcione. Para eso hace falta un GET /productos exitoso, que es lo que hace la prueba de humo.

23.4 Lo que la fachada no expone

- Documentación interactiva de FastAPI (/docs, /openapi.json) no tiene recurso en API Gateway. Sigue disponible dentro del clúster si se hace un port-forward al Service.
- No hay paginación, filtros ni orden en los listados.
- No hay control de concurrencia (no hay versión del ítem ni ConditionExpression). Dos PUT simultáneos: gana el último PutItem.

24. Seguridad e identidad

Mecanismo	Qué protege	Límite en esta versión
KMS	Secretos de Kubernetes en etcd	No cifra DynamoDB con CMK propia
Rol del clúster	Llamadas del plano de control	Policy administrada de EKS
Rol de nodo bootstrap	Pull de imágenes, CNI, SSM	Lo asume la instancia, no el pod
Rol de nodo Karpenter	Igual, en nodos elásticos	Igual separación
IRSA Karpenter	Crear y terminar instancias	Acciones EC2 amplias sobre *
IRSA FastAPI	Tablas del proyecto	No autoriza al usuario HTTP
NLB interno	Oculto el ingress	El API server sigue con endpoint público
API Gateway	Única URL de negocio	AuthorizationType: NONE
Sin llaves en el manifiesto	Evita filtrar accesos de larga duración	La sesión de la máquina que despliega sí tiene poder de administrador de estos stacks

La cadena de confianza del pod es: token proyectado del ServiceAccount, proveedor OIDC, Assume RoleWithWebIdentity, credenciales temporales, firma de boto3. Rotar la CMK no rota esas credenciales. Borrar el rol o quitar la anotación hace fallar DynamoDB con acceso denegado en la siguiente llamada, mientras /health sigue en verde. Por eso un health check de negocio debería leer la tabla; el actual no lo hace.

El tráfico de gestión (`kubectl`) usa el endpoint del clúster y IAM, no la API de productos. `make kubeconfig` ejecuta `aws eks update-kubeconfig` con el nombre y la región de `vars.yaml`. Quien no pueda asumir permisos `eks:DescribeCluster` no obtiene el kubeconfig aunque conozca el nombre.

25. Pipeline de despliegue

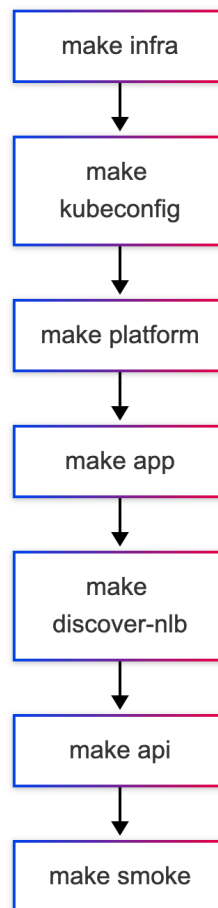


Figura 8: Orden de los objetivos de Make

```
make infra           # CloudFormation hasta el stack ec2
make kubeconfig      # kubectl contra el clúster
make platform        # Karpenter, Istio, NodePool, Gateway
make app             # FastAPI
make discover-nlb    # escribe NlbArn y NlbDnsName
make api             # API Gateway y VPC Link
make smoke           # GET de productos y clientes
```

`make all` ejecuta esa secuencia, incluyendo el descubrimiento del NLB entre la aplicación y la fachada. `make validate` corre `scripts/validate-cfn.sh` si se quiere lint de las plantillas antes de desplegar. `make sync-vars` y `scripts/fetch-outputs.sh` pueden refrescar salidas; no sustituyen

el descubrimiento del NLB.

25.1 Qué hace cada fase

infra. Seis `aws cloudformation deploy` en orden, leyendo salidas entre ellos. El más lento es EKS: el plano de control y el node group tardan bastante más que la VPC o las tablas. Repetir `make infra` cuando no hay cambios termina en `changeset` vacío y código de salida correcto.

kubeconfig. Apunta el contexto local al clúster. No instala nada en AWS.

platform. Exige `kubectl`, `helm` e `istioctl`. Instala o actualiza Karpenter, instala Istio con el operador reducido, aplica el Service del NLB y el Gateway, renderiza y aplica Karpenter. El NLB puede tardar unos minutos en pasar a activo y publicar hostname. Hasta ese momento `discover-nlb` no tiene DNS.

app. Renderiza manifiestos, aplica namespace y ServiceAccount, elige ConfigMap o imagen, aplica Service y VirtualService, y espera el rollout. La primera vez, el pull de `python:3.12-slim` y el `pip install` consumen la mayor parte de los 180 segundos.

discover-nlb. Si `kubectl` ve el Service `istio-ingressgateway-internal`, lee `status.loadBalancer.ingress[0].hostname` y busca ese DNS en ELBv2 para obtener el ARN. Si no, toma el primer NLB interno de la región. Escribe ambas claves en `vars.yaml` mediante `scripts/parse-vars.py`. En una cuenta con varios NLB internos, el fallback por “el primero” puede apuntar al balanceador equivocado: el camino fiable es el hostname del Service.

api. Si las claves del NLB siguen vacías, llama otra vez al descubrimiento. Luego despliega el stack `api`.

smoke. Lee `InvokeUrl`, hace `GET /productos` hasta cinco veces con 15 segundos de pausa y exige HTTP 200. Después hace un solo `GET /clientes` y también exige 200. Imprime los primeros 200 caracteres del cuerpo. Un 503 aquí suele ser el pod todavía no Ready o el VPC Link todavía propagándose.

25.2 Render de manifiestos

`scripts/render-k8s.sh` exporta región, clúster, tablas, rol del pod, rol de nodo Karpenter y SG. Lee los dos ARN o ids desde el stack EKS con `aws cloudformation describe-stacks`. Aplica `envsubst` a los YAML que contienen `${...}` y copia el NodePool sin cambios. La salida es `.rendered/k8s/`. Aplicar los YAML de `k8s/` a mano, sin renderizar, deja el placeholder del rol y el pod no puede asumir nada.

26. Operación diaria

Comprobaciones útiles después de `make all`, o cuando la prueba de humo falla:

```

kubectl get nodes
kubectl get pods -A
kubectl get pods -n api-mercado
kubectl logs -n api-mercado deploy/fastapi -c api
kubectl get svc -n istio-system istio-ingressgateway-internal
kubectl describe ec2nodeclass default
kubectl get nodepool,nodeclaim

```

El contenedor de la API se llama `api`. Si el sidecar está inyectado, los logs del Deployment sin `-c api` mezclan Envoy. Para ver el motivo de un Pending: `kubectl describe pod` en `api-mercado` y los logs del controlador en `kube-system`. Un NodeClaim que no pasa a Ready suele ser AMI, subnet sin el tag, o Spot sin capacidad para `t3a.small` y `t3a.medium` en esas dos zonas.

Actualizar solo el código Python, en modo ConfigMap:

```
make app
```

Eso recrea el ConfigMap y reinicia el Deployment. No toca la VPC ni API Gateway. Cambiar un puerto o un prefijo sí exige alinear VirtualService, Service y, si la ruta es nueva, la plantilla de API Gateway seguida de `make api`.

Actualizar una plantilla ya desplegada es volver a ejecutar `make infra` o el `aws cloudformation deploy` de ese stack. Hay cambios que reemplazan recursos (CIDR de la VPC, nombre del node group, nombre de tabla). En DynamoDB, renombrar la tabla crea una tabla nueva vacía y puede intentar borrar la anterior: hay que tratar el nombre como inmutable una vez que hay datos.

27. Pruebas

27.1 Prueba de humo automatizada

`make smoke` cubre el camino de lectura público. No crea datos. Sobre tablas vacías, 200 con `{"items": []}` es éxito. 403 desde DynamoDB indica un rol IRSA mal anotado o una policy que no incluye el ARN real. 503 indica que el upstream no tiene endpoints. Un timeout de `curl` indica que el VPC Link o el NLB no enruta, no que el pod esté mal: el pod se comprueba con `kubectl`.

27.2 Prueba manual del contrato

Sustituir `INVOKE` por la salida `InvokeUrl`:

```

INVOKE=$(aws cloudformation describe-stacks \
  --stack-name api-mercado-dev-api \
  --query "Stacks[0].Outputs[?OutputKey=='InvokeUrl'].OutputValue" \
  --output text)

curl -sS -X POST "$INVOKE/productos" \

```

```
-H 'content-type: application/json' \
-d '{"nombre":"Arroz 1kg","precio":"1,25","stock":"10"}'
```

```
curl -sS "$INVOKE/productos"
```

```
curl -sS -X POST "$INVOKE/clientes" \
-H 'content-type: application/json' \
-d '{"nombre":"Ana Ruiz","fechaNacimiento":"12/03/1994"}'
```

El precio con coma y el stock como texto ejercitan los validadores. La fecha con barras debe volver como 1994-03-12. Un segundo POST de producto debe generar otro `productoId`. DELETE de ese id debe devolver 204 y el GET siguiente 404.

27.3 Qué no está automatizado

No hay pruebas unitarias de `main.py` en el repositorio, ni pruebas de carga, ni un chequeo que cree y borre un ítem dentro de `make smoke`. La validación de plantillas (`make validate`) revisa sintaxis CloudFormation; no despliega. La generación del PDF (`make docs-pdf`) comprueba que los diagramas renderizan y que Pandoc termina; no comprueba AWS.

28. Costos en el laboratorio

Los importes exactos cambian y se consultan en la lista de precios de la región. Los generadores de costo de este diseño, de mayor a menor impacto típico en un laboratorio encendido todo el día, son:

Generador	Por qué aparece	Cómo lo contiene el diseño
NAT Gateway	Cargo por hora y por gigabyte procesado, aunque casi no haya tráfico de clientes	Un solo NAT, no uno por zona
Plano de control EKS	Cargo por hora del clúster	Un solo clúster
VPC Link	Cargo por hora del enlace de REST API	Un solo link
EC2 Spot	Nodo bootstrap y nodos Karpenter	Tipos pequeños, tope de 4 vCPU, consolidación
API Gateway	Por millón de llamadas	Tráfico de laboratorio
DynamoDB on-demand	Por lectura y escritura	Sin capacidad provisionada ociosa
Direcciones y registros	IP elástica del NAT, logs si se activan fuera de este repo	No hay logs de flujo de VPC en la plantilla

Generador	Por qué aparece	Cómo lo contiene el diseño
KMS	La clave y las peticiones de cifrado de secretos	Una CMK

Apagar el laboratorio de verdad implica borrar los stacks en orden inverso (API, plataforma Kubernetes, luego EKS, IAM, tablas, KMS, VPC) o borrar el clúster y el NAT, que son los relojes caros. Borrar solo los pods no detiene el cargo de EKS ni el del NAT. `vars.yaml` no incluye un objetivo `make destroy`: la destrucción queda como operación consciente en la consola o en la CLI, para no borrar datos con un objetivo fácil de disparar.

El NAT procesa el pull de imágenes y las llamadas a DynamoDB y STS. Cada `make app` en modo ConfigMap vuelve a instalar paquetes si el contenedor es nuevo, y eso es tráfico de salida. Pasar a una imagen en ECR no elimina el NAT (el nodo sigue saliendo a ECR por él, salvo que se añada un VPC endpoint), pero sí elimina el `pip install` de cada arranque.

29. Incidentes frecuentes

Síntoma	Causa habitual	Qué hacer
<code>aws: command not found</code>	CLI fuera del PATH	Incluir el directorio de la CLI en PATH
Credenciales ausentes o perfil distinto	<code>AwsProfile</code> no coincide con la sesión	Ajustar el perfil o la variable <code>AWS_PROFILE</code>
EKS rechaza la versión	Versión fuera de soporte	Subir <code>KubernetesVersion</code> y el AMI compatible
Node group 409 al cambiar AMI	El nombre del grupo ya existe	El nombre con sufijo <code>al2023</code> evita el choque; no reutilizar el nombre viejo
Istio en Pending o timeout	Poca memoria en el bootstrap	Mantener el <code>IstioOperator</code> reducido y <code>t3a.medium</code>
Helm de Karpenter atascado	Release <code>pending-*</code>	El script lo desinstala; si se hizo a mano, <code>helm uninstall</code> en <code>kube-system</code> y <code>replicas=1</code>
CRD <code>EC2NodeClass</code> v1 no existe	Chart 0.37	Usar <code>karpenter.k8s.aws/v1beta1</code>
Pods Pending tras el bootstrap	Sin tag de subnet o sin Spot	Revisar <code>karpenter.sh/discovery</code> y la oferta Spot
<code>make app</code> no termina	<code>pip</code> o pull lento	Ver logs del contenedor <code>api</code> ; la sonda espera 60 s

Síntoma	Causa habitual	Qué hacer
<code>make smoke HTTP 503</code>	Sin endpoints o VPC Link nuevo	<code>kubect1 get pods -n api-mercado</code> y repetir tras unos minutos
HTTP 200 en <code>/health</code> y 403 en datos	IRSA	Anotación del SA, subject del rol y ARN de las tablas
<code>discover-nlb</code> elige otro balanceador	Fallback al primer NLB interno	Esperar el hostname del Service y volver a descubrir
Imagen local no arranca en el nodo	Docker build no se subió	Modo ConfigMap, o URI de ECR en <code>FASTAPI_IMAGE</code>
Listado incompleto	<code>Scan</code> de más de 1 MB	Esperable; falta paginar en código
PDF con figuras enormes, cortadas o en tira estrecha	PNG sin densidad o diagrama demasiado alto	<code>make docs-pdf</code> (usa <code>--size</code> , fondo blanco y DPI acotado)

30. Riesgos y trabajo futuro

Riesgo	Impacto	Mitigación presente	Mejora
API abierta	Cualquiera con la URL modifica datos	Solo laboratorio	Autorizador en los métodos
Un solo NAT	Corte de salida y costo fijo	Documentado	NAT por zona, o VPC endpoints
Spot en la única réplica	Corte durante interrupción	Aceptado en dev	Dos réplicas y PDB, o On-Demand
<code>Scan</code> sin paginar	Listados truncados	Catálogo pequeño	Bucle de <code>LastEvaluatedKey</code> o GSI
Policy de Karpenter amplia	Un controlador comprometido mueve EC2	Rol separado del de la app	Acotar recursos cuando el proveedor lo permita
Endpoint público del API server	Superficie de gestión	IAM de AWS	CIDR de acceso público

Riesgo	Impacto	Mitigación presente	Mejora
Sin CI	Un cambio se despliega a mano	Scripts idempotentes	Pipeline con <code>make validate</code> y <code>make docs-pdf</code>
ConfigMap con pip	Arranque lento y variable	Sondas holgadas	Imagen en ECR

El orden recomendado para endurecer el laboratorio, si deja de ser un laboratorio, es: autorización en API Gateway, imagen inmutable en ECR, paginación, segundo NAT o endpoints de VPC, y solo entonces más réplicas y observabilidad. Añadir réplicas sobre una API abierta multiplica el daño, no la madurez.

31. Estructura del repositorio

```

iac_api_mercado/
  vars.yaml           Parámetros globales
  Makefile            infra, platform, app, docs-pdf
  template.yaml       Stack maestro opcional (S3)
  vpc/ kms/ iam/ dynamodb/ eks/ ec2/ api_gateway/
  k8s/karpenter/      EC2NodeClass y NodePool
  k8s/istio/          IstioOperator y Gateway NLB
  k8s/app/            Namespace, SA, Deployment, Service, VirtualService
  k8s/fastapi/        main.py y requirements.txt
  scripts/deploy.sh    Fases de despliegue
  scripts/render-k8s.sh envsubst de manifiestos
  scripts/discover-nlb.sh ARN y DNS del NLB interno
  scripts/build-docs.sh Diagramas y PDF
  docs/PROYECTO.md     Este informe
  docs/diagramas/      Fuentes Mermaid
  docs/pdf-header.tex  Corte de lineas en bloques de codigo
  docs/mermaid-config.json Tema base y useMaxWidth en falso

```

`.rendered/` y `docs/media/` se generan. El PDF también se genera y está ignorado por Git, igual que los PNG: la fuente de verdad es el Markdown y los `.mmd`.

32. Generación de este PDF

Los diagramas usan el tema `base`. Cada `.mmd` incluye la directiva de tema y `docs/mermaid-config.json` repite el tema con `flowchart.useMaxWidth` en falso. Ese ajuste evita que Mermaid fije el SVG al ancho de la ventana del navegador headless y que Puppeteer exporte un lienzo de varios miles de píxeles. El fondo de exportación es blanco: un PNG transparente se ve mal o con la caja deformada al incrustarlo en el PDF.

`scripts/build-docs.sh` hace, por cada diagrama:

1. Render con `--size 1400` (lado mayor) y escala 2, fondo blanco. Las opciones antiguas `-w` y `-H` ya no existen en Mermaid CLI; sin `--size` el lienzo queda sin tope y LaTeX lo dibuja fuera de la página o reporta un error de dimensión.
2. Si el PNG supera 1600 píxeles de ancho o 1400 de alto, lo reescala con `sips` (macOS) o con ImageMagick.
3. Calcula la densidad para que el tamaño impreso quepa en 14,5 cm de ancho y 11 cm de alto, sin ampliar la imagen por encima de su tamaño natural.

Pandoc envuelve cada figura y solo la reduce si todavía no cabe en la caja de texto. `docs/pdf-header.tex` activa el corte de líneas en los bloques de código. No fuerza un ancho de `graphicx`, porque ese ajuste estira los diagramas hasta ocupar media página.

Requisitos locales:

```
brew install pandoc mermaid-cli tectonic
```

Generar:

```
make docs-pdf
```

Salida: `docs/PROYECTO.pdf`. Si no hay motor LaTeX, el script escribe `docs/PROYECTO.html` y termina con error para que no se confunda el HTML con el entregable. La hoja `docs/pdf.css` limita también las imágenes de ese HTML.

Conviene regenerar el PDF cuando cambie un diagrama, un puerto, un prefijo o el orden de los stacks. El Markdown es la fuente; no se edita el PDF a mano.

33. Glosario

Término	En este proyecto
IRSA	IAM Roles for Service Accounts. El pod asume un rol mediante el OIDC del clúster
VPC Link	Recurso de API Gateway REST que integra con un NLB de la VPC
NLB	Network Load Balancer. Aquí es interno y recibe TCP en el puerto 80

Término	En este proyecto
Node group bootstrap	Managed node group pequeño que sostiene Karpenter e Istio
NodePool	Objeto de Karpenter que declara restricciones de capacidad
EC2NodeClass	Objeto de Karpenter que declara AMI, rol, subnets y grupos de seguridad
VirtualService	Reglas HTTP de Istio desde un Gateway hacia un Service
Stage	Nombre de despliegue de API Gateway (dev), prefijo de la URL pública
ConfigMap de código	Manera de enviar <code>main.py</code> al pod sin un registro de imágenes
Spot	Capacidad EC2 con descuento que AWS puede reclamar
CMK	Clave de KMS administrada por el cliente, usada para los secretos de EKS

34. Anexo — lectura de un despliegue correcto

Esta sección fija qué debe verse cuando el laboratorio quedó bien, para no confundir un arranque lento con un fallo de arquitectura. Sirve también como lista de cierre antes de dar por válido el entorno.

34.1 Después de la red y de los datos

La VPC existe con cuatro subnets y un NAT. Las dos privadas llevan las etiquetas de balanceador interno y de descubrimiento de Karpenter. Las tablas responden a `DescribeTable` y están en modo bajo demanda. La clave KMS tiene alias `alias/{Project}-{Environment}-eks` y la rotación habilitada. Todavía no hay pods ni URL pública de negocio: es el estado normal al terminar `make infra`.

34.2 Después del clúster y de la plataforma

`kubectl get nodes` muestra al menos el nodo bootstrap en Ready, con AMI de Amazon Linux 2023. En `kube-system` están el pod de Karpenter (una réplica), `istiod` y el `ingress`. El Service `istio-ingressgateway-internal` tiene un hostname en `status.loadBalancer.ingress`. Hasta que ese hostname existe, no hay NLB que apuntar y `make api` no debe inventar un ARN.

El `EC2NodeClass default` referencia el rol de nodo Karpenter y el grupo de seguridad del clúster. El `NodePool spot-workloads` limita la CPU a 4 y solo admite los dos tipos `t3a`. Si la aplicación cabe en el bootstrap, es válido que no aparezca un segundo nodo: Karpenter no crea instancias por anticipado.

34.3 Después de la aplicación

En el namespace `api-mercado` hay un pod `fastapi` en Running y Ready. El contenedor `api` ya pasó el `pip install` si el modo es ConfigMap. El ServiceAccount tiene la anotación del rol. Un GET interno a `/health` responde `status: ok`. Eso no prueba DynamoDB. La prueba que sí lo prueba es GET `/productos` a través de la URL de API Gateway, que es lo que repite `make smoke` hasta cinco veces.

34.4 Después de la fachada

El stack de API Gateway publica `InvokeUrl`. Un POST de producto devuelve 201 y un UUID. El mismo id en GET devuelve el ítem. DELETE devuelve 204. La fecha de un cliente aceptada como DD/MM/YYYY se lee después como YYYY-MM-DD. Si el listado responde 200 con `items` vacío, el camino está sano y simplemente no hay datos.

34.5 Señales que no deben ignorarse

Un pod Ready con errores `AccessDenied` en el log al llamar a DynamoDB es un fallo de IRSA, no de red. Un NLB activo con 503 en la URL pública es un fallo de endpoints o de VirtualService. Un `make discover-nlb` que escribe un ARN cuya DNS no coincide con el Service está apuntando a otro balanceador de la cuenta. Un PDF regenerado con figuras cortadas o más anchas que el texto indica que se volvió a exportar el diagrama sin `--size` o sin recalcular la densidad: hay que correr `make docs-pdf` completo, no incrustar el PNG a mano.

35. Referencias

- Amazon EKS, guía de usuario: planos de control, node groups, IRSA y addons.
- Amazon API Gateway, integración privada de REST API mediante VPC Link y NLB.
- AWS CloudFormation, función intrínseca `Fn::Cidr` y despliegues con `aws cloudformation deploy`.
- Karpenter, documentación del chart 0.37 y de los CRD `v1beta1` (`NodePool`, `EC2NodeClass`).
- Istio, IstioOperator, Gateway y VirtualService.
- FastAPI y Pydantic v2, modelos y códigos de validación.
- boto3, recurso de DynamoDB (`Table.put_item`, `get_item`, `scan`, `delete_item`).

Las versiones concretas que usa el laboratorio están en `vars.yaml` y en `k8s/fastapi/requirements.txt`, no en estas referencias generales.

*Documento elaborado por **Marlon Ernesto Figueroa Fuentes**. Actualizar `docs/PROYECTO.md` y ejecutar `make docs-pdf` cuando cambie la arquitectura, el contrato o el tamaño de las figuras.*